PROGETTO C# · PROGRAMMAZIONE A OGGETTI

Simulatore di una Smart Home

Traccia di esercitazione · C# / .NET / Windows Forms · Livello intermedio-avanzato

Obiettivo didattico
Progettare un'applicazione desktop capace di coordinare dispositivi intelligenti diversi, applicando in modo motivato classi, sottoclassi, classi astratte, interfacce, polimorfismo ed eventi. La logica del dominio deve essere separata dall'interfaccia grafica.

1. Scenario

Un'azienda vuole realizzare il prototipo di un software di controllo domestico. In una casa sono presenti stanze e dispositivi di tipologie differenti. L'utente deve poter visualizzare lo stato dei dispositivi, inviare comandi, ricevere notifiche e attivare semplici automatismi. La simulazione è interamente locale: non sono richiesti hardware, servizi cloud o database.

2. Requisiti funzionali

- Abitazione e ambienti. Consentire la gestione di almeno tre stanze. Ogni dispositivo appartiene a una stanza ed è identificabile tramite un codice univoco e un nome leggibile.
- Luci intelligenti. Accensione e spegnimento, regolazione della luminosità da 0 a 100 e visualizzazione dello stato corrente.
- Termostati. Impostazione di una temperatura desiderata entro limiti validi; simulazione della temperatura rilevata e segnalazione al raggiungimento della soglia impostata.
- Tapparelle motorizzate. Apertura, chiusura e posizionamento percentuale (0 = chiusa, 100 = aperta). Non devono essere accettate posizioni fuori intervallo.
- Sensori. Implementare almeno due tipologie tra movimento, temperatura, fumo e apertura porte. I sensori producono letture o segnalazioni, ma non devono esporre comandi inappropriati.
- Gestione dei dispositivi. Aggiunta, rimozione, ricerca per stanza/tipologia e visualizzazione dei dettagli e dello stato aggiornato.
- Automazioni. Realizzare almeno due regole: ad esempio, movimento rilevato → accensione di una luce; rilevazione fumo → allarme; temperatura superiore a una soglia → notifica.
- Registro attività. Mostrare in ordine cronologico le principali operazioni e notifiche prodotte durante la simulazione.

3. Vincoli architetturali obbligatori

- Classi e incapsulamento. Modellare stanze, dispositivi e regole con responsabilità precise. Proteggere lo stato mediante proprietà e metodi appropriati; evitare campi pubblici modificabili.
- Classe astratta. Definire una base astratta comune per i dispositivi, che raccolga identità, collocazione e comportamenti effettivamente condivisi. Motivare quali membri sono astratti, virtuali o concreti.
- Ereditarietà e polimorfismo. Prevedere sottoclassi per dispositivi differenti. La gestione generale dell'abitazione deve operare sui dispositivi senza ricorrere a grandi catene di if/switch per ciascun

tipo.

- Interfacce. Progettare almeno due interfacce basate sulle capacità, per esempio dispositivi attivabili, regolabili o capaci di fornire misurazioni. Una capacità va esposta solo dai dispositivi che la supportano.
- Eventi C#. Usare eventi e delegati (event, EventHandler o EventHandler<T>) per notificare cambiamenti di stato o segnalazioni. I componenti interessati devono sottoscriversi agli eventi senza dipendere direttamente dai controlli WinForms.
- Separazione dei livelli. La logica applicativa deve restare in classi indipendenti dal form; il form visualizza lo stato, raccoglie input e invoca operazioni pubbliche. Vietato collocare tutte le regole di business nei gestori dei pulsanti.

4. Interfaccia Windows Forms

Realizzare almeno una finestra principale che includa:

- Elenco delle stanze e dei dispositivi, con possibilità di selezione e filtro.
- Pannello dei dettagli del dispositivo selezionato e comandi coerenti con le sue capacità (es. pulsanti, TrackBar, NumericUpDown).
- Comandi per creare o rimuovere dispositivi dalla simulazione.
- Area notifiche/registro aggiornata in risposta agli eventi, senza refresh manuale.
- Controlli per simulare rilevazioni dei sensori e verificare l’attivazione delle automazioni.

5. Scenari di verifica obbligatori

- Aggiungendo due dispositivi con caratteristiche diverse, l’applicazione li mostra e consente solo le operazioni compatibili.
- Spostando una tapparella al 70%, la posizione visualizzata diventa 70%; un valore non valido è rifiutato o gestito correttamente.
- Quando il sensore di movimento rileva presenza, l’automazione accende la luce associata e produce una voce nel registro.
- Il termostato emette una notifica al verificarsi della condizione di soglia; la WinForms si aggiorna tramite la gestione dell’evento.
- Rimuovendo un dispositivo, non restano sottoscrizioni o riferimenti inutili che provochino notifiche indesiderate.

6. Consegna

- Soluzione Visual Studio compilabile e avviabile, con codice sorgente C# e progetto Windows Forms.
- Diagramma UML delle classi principali, con relazioni di ereditarietà, implementazioni di interfacce e associazioni.
- Breve relazione (1–2 pagine) che spieghi le scelte progettuali, il flusso di almeno un evento e il funzionamento delle automazioni.
- Istruzioni essenziali per eseguire e provare i cinque scenari di verifica.

7. Criteri di valutazione suggeriti

Area	Peso
Modello OOP, incapsulamento ed ereditarietà	30%
Interfacce, classi astratte e polimorfismo	25%
Eventi, automazioni e disaccoppiamento	25%
Usabilità WinForms, verifiche e documentazione	20%

8. Estensioni facoltative

Salvataggio e caricamento della configurazione in JSON; scenari preimpostati (es. "Notte" o "Fuori casa"); consumo energetico simulato; storico delle letture dei sensori; implementazione di test unitari per le regole di dominio.

Nota: la traccia definisce comportamenti e vincoli, ma non impone un diagramma delle classi prestabilito: individuare le astrazioni corrette fa parte dell’esercizio.